

Manual de Laboratorio de Matemáticas

Grado en Ingeniería Matemática

Borrador docente

Curso 2026/27

Propósito del manual

Este manual acompaña la asignatura *Laboratorio de Matemáticas*, de primer curso del Grado en Ingeniería Matemática. La asignatura tiene 6 ECTS, 15 horas de clase magistral y 45 horas de prácticas presenciales. Su finalidad es introducir el uso de herramientas computacionales para resolver, documentar y comunicar problemas matemáticos.

El enfoque principal del manual es el ecosistema R: programación básica, cálculo numérico elemental, tratamiento de datos, representación gráfica y modelización estadística inicial. La edición científica se trabaja con LaTeX, y los cuadernos reproducibles se presentan mediante Markdown, R Markdown o Quarto.

Cómo usar este documento

Cada bloque combina una parte conceptual breve con ejemplos reproducibles y una práctica de laboratorio. Las prácticas están pensadas para sesiones de 3 horas. El manual puede adaptarse a grupos con distinto nivel inicial ampliando o reduciendo los apartados marcados como extensión.

Convenciones

- El símbolo `>` identifica comandos ejecutados en la consola de R.
- Los fragmentos de código se pueden copiar en un script `.R`, salvo que se indique otra cosa.
- Las entregas prácticas deben incluir código, resultados y una explicación breve de las decisiones tomadas.
- Las soluciones no se valoran solo por dar un número correcto: importan la organización del proyecto, la claridad del código y la interpretación matemática.

Índice general

1. Organización de la asignatura	1
1.1. Resultados de aprendizaje	1
1.2. Distribución temporal	1
1.3. Criterios de evaluación recomendados	2
2. Entorno de trabajo	3
2.1. R, RStudio y proyectos	3
2.2. La carpeta de trabajo	4
2.3. Primeros comandos	4
2.4. Paquetes y bibliotecas	5
2.5. Ayuda integrada	5
2.6. Información de la sesión	5
2.7. Flujo de trabajo recomendado	5
3. Objetos, tipos de datos y expresiones	7
3.1. Objetos y asignación	7
3.2. Tipos básicos	7
3.3. Operadores y precedencia	8
3.4. Conversión y reciclaje	8
3.5. Valores especiales y datos ausentes	9
3.6. Errores de redondeo	9
3.7. Atributos e inspección	10
3.8. Lista de comprobación	10
4. Funciones	11
4.1. Funciones predefinidas	11
4.2. Funciones propias	11
4.3. Ejemplo: distancia euclídea	11
5. Control de flujo	12
5.1. Condicionales	12
5.2. Bucles	12
5.3. Ejemplo: algoritmo de Euclides	12
6. Vectores, matrices y estructuras de datos	14
6.1. Vectores	14
6.2. Secuencias	14
6.3. Matrices	14
6.4. Tablas de datos	14

7. Representación gráfica	16
7.1. Gráficos base	16
7.2. ggplot2	16
7.3. Criterios para un buen gráfico	16
8. Cálculo numérico elemental	17
8.1. Evaluación de funciones	17
8.2. Búsqueda de raíces	17
8.3. Derivación numérica	17
8.4. Integración numérica	17
9. Simulación	19
9.1. Números pseudoaleatorios	19
9.2. Monte Carlo para estimar π	19
9.3. Error de simulación	19
10.Tratamiento de datos y estadística inicial	20
10.1. Importación de datos	20
10.2. Resumen descriptivo	20
10.3. Regresión lineal	20
11.Edición científica con LaTeX	21
11.1. Estructura mínima	21
11.2. Fórmulas	21
11.3. Tablas	21
11.4. Buenas prácticas	22
12.Markdown, R Markdown y Quarto	23
12.1. Markdown	23
12.2. Cuadernos reproducibles	23
12.3. Estructura recomendada de un informe	23
I Prácticas de laboratorio	24
13.Guía general de prácticas	25
13.1. Formato común de entrega	25
13.2. Rúbrica común	25
14.Práctica 1. Entorno de trabajo y primer proyecto	26
14.1. Actividad	26
15.Práctica 2. Cálculo elemental y precisión numérica	27
15.1. Actividad	27
16.Práctica 3. Funciones propias	28
16.1. Actividad	28
17.Práctica 4. Condicionales, bucles y algoritmos elementales	29
17.1. Actividad	29
18.Práctica 5. Vectores y matrices	30
18.1. Actividad	30

19.Práctica 6. Datos en tablas	31
19.1. Actividad	31
20.Práctica 7. Visualización de funciones y datos	32
20.1. Actividad	32
21.Práctica 8. Raíces y aproximación numérica	33
21.1. Actividad	33
22.Práctica 9. Derivación e integración numérica	34
22.1. Actividad	34
23.Práctica 10. Simulación y Monte Carlo	35
23.1. Actividad	35
24.Práctica 11. Estadística descriptiva	36
24.1. Actividad	36
25.Práctica 12. Regresión lineal	37
25.1. Actividad	37
26.Práctica 13. Documento científico en LaTeX	38
26.1. Actividad	38
27.Práctica 14. Informe reproducible con R Markdown o Quarto	39
27.1. Actividad	39
28.Práctica 15. Proyecto integrador	40
28.1. Propuesta	40
28.2. Requisitos mínimos	40
A. Referencia rápida de R	41
A.1. Objetos y operaciones	41
A.2. Funciones	41
A.3. Control de flujo	41
B. Plantilla de informe de práctica	42
C. Bibliografía inicial	43

Capítulo 1

Organización de la asignatura

1.1. Resultados de aprendizaje

Al completar la asignatura, el estudiante debe ser capaz de:

- utilizar R como entorno de trabajo para resolver problemas matemáticos sencillos;
- escribir programas con variables, funciones, condicionales, bucles y estructuras de datos;
- aplicar herramientas computacionales a cálculo, simulación, datos y estadística inicial;
- generar gráficos y comunicar resultados cuantitativos de forma clara;
- elaborar documentos científicos básicos en LaTeX;
- crear cuadernos reproducibles con Markdown, R Markdown o Quarto;
- organizar proyectos, scripts, datos y resultados de forma trazable.

1.2. Distribución temporal

La siguiente planificación distribuye las 15 horas magistrales y las 45 horas prácticas en 15 semanas. Cada semana contiene 1 hora magistral y 3 horas de laboratorio.

Semana	Clase magistral	Práctica de laboratorio
1	Entorno de trabajo y proyectos	Instalación, RStudio/Posit y primer proyecto
2	Objetos, tipos y expresiones	Cálculo elemental y scripts reproducibles
3	Funciones y ayuda integrada	Funciones propias y documentación mínima
4	Condicionales e iteraciones	Algoritmos elementales y comprobaciones
5	Vectores y matrices	Operaciones vectorizadas y álgebra computacional
6	Listas y tablas de datos	Importación, limpieza y exploración de datos
7	Gráficos base y <code>ggplot2</code>	Visualización de funciones y datos
8	Cálculo numérico elemental	Raíces, derivadas e integrales aproximadas

9	Simulación	Experimentos aleatorios y estimación por Monte Carlo
10	Estadística descriptiva	Resúmenes, distribuciones y comparación de grupos
11	Modelización estadística inicial	Regresión lineal e interpretación de modelos
12	LaTeX científico	Fórmulas, tablas, figuras y bibliografía
13	Markdown y cuadernos reproducibles	Informe reproducible con R Markdown o Quarto
14	Proyecto integrador	Desarrollo guiado de un informe computacional
15	Presentación y revisión	Entrega, defensa breve y mejora final

1.3. Criterios de evaluación recomendados

La guía docente permite combinar evaluación continua y examen final, cada uno con peso entre el 40 % y el 60 %. Una concreción coherente para esta asignatura es:

- Evaluación continua, 50 %: prácticas semanales, informes reproducibles, ejercicios de programación y participación técnica en laboratorio.
- Examen final, 50 %: resolución individual de problemas de programación, cálculo, análisis de datos y edición científica.

En las prácticas se recomienda valorar:

- corrección matemática y computacional;
- claridad y simplicidad del código;
- uso adecuado de funciones y estructuras de datos;
- reproducibilidad del trabajo;
- interpretación escrita de resultados;
- presentación formal del informe.

Capítulo 2

Entorno de trabajo

Objetivos de aprendizaje

Al finalizar este capítulo, el estudiante debe ser capaz de:

- distinguir entre la consola, un script y un proyecto de R;
- organizar los archivos de una práctica de forma reproducible;
- instalar y cargar paquetes sin confundir ambas operaciones;
- consultar la ayuda integrada y obtener información sobre la sesión de trabajo.

2.1. R, RStudio y proyectos

R es un lenguaje de programación orientado al cálculo estadístico, el análisis de datos y la representación gráfica. RStudio o Posit Desktop proporciona un entorno integrado para escribir scripts, ejecutar código, consultar ayuda, visualizar gráficos y organizar proyectos.

Conviene distinguir tres elementos que a menudo se mezclan al comenzar:

- La **consola** ejecuta instrucciones inmediatamente. Es útil para explorar, pero no conserva por sí sola una explicación completa del trabajo.
- Un **script** es un archivo de texto, normalmente con extensión `.R`, que contiene instrucciones en un orden explícito. Es el lugar habitual para guardar el código de una práctica.
- Un **proyecto** asocia una carpeta con el trabajo de R. Al abrirlo, RStudio puede recuperar la carpeta del proyecto y facilita que las rutas relativas sean estables.

La consola sirve para probar una expresión; cuando una prueba es útil, se debe pasar al script con un comentario que explique su propósito. No es recomendable basar una entrega en objetos creados accidentalmente en el espacio de trabajo.

Un proyecto bien organizado evita pérdidas de información y facilita que otra persona pueda reproducir los resultados. Una estructura básica es:

```
laboratorio-matematicas/  
  datos/  
  scripts/  
  informes/  
  figuras/  
  resultados/  
  README.md
```


La carpeta `datos/` contiene los datos originales y no debe mezclarse con resultados transformados. Los scripts deben poder ejecutarse desde el principio y generar los resultados a partir de los archivos que el proyecto incluye. Si un dato se descarga de una fuente externa, se debe registrar su procedencia, fecha de descarga y transformación aplicada.

2.2. La carpeta de trabajo

Las rutas de archivos dependen de la carpeta de trabajo actual. En un proyecto de RStudio, la solución preferida es utilizar rutas relativas al directorio del proyecto, no rutas absolutas ligadas a un ordenador concreto.

```
getwd()
list.files()
list.files("datos")

archivo <- file.path("datos", "mediciones.csv")
archivo
```

La función `file.path()` construye rutas respetando el sistema operativo. Esta práctica es más robusta que escribir manualmente separadores como `/` o `.`.

Observación

Evita comandos como `setwd(C:/Users/...)` en los scripts que se entregan. Funcionan solo en el ordenador donde se escribieron y dificultan la reproducción del análisis.

2.3. Primeros comandos

```
2 + 3
sqrt(2)
sin(pi / 4)
log(exp(1))

x <- 5
y <- 2 * x + 1
y
```

R distingue entre una expresión que se evalúa y una asignación que guarda el resultado. Escribir el nombre de un objeto en una línea independiente permite inspeccionarlo; añadir un punto y coma no hace que un script sea más claro.

Los comentarios comienzan con `#`. Deben explicar una decisión o separar partes lógicas del programa, no repetir literalmente lo que ya dice el código.

```
# Pendiente de una recta que pasa por dos puntos
x1 <- 1
y1 <- 3
x2 <- 4
y2 <- 9
pendiente <- (y2 - y1) / (x2 - x1)
pendiente
```

2.4. Paquetes y bibliotecas

La instalación de un paquete se hace normalmente una sola vez por versión de R. Cargarlo con `library()` es una operación distinta y suele hacerse al comienzo de cada sesión o script.

```
# Solo si el paquete todavía no está instalado
install.packages("ggplot2")

# Al comenzar un script que utiliza el paquete
library(ggplot2)
```

Para comprobar si un paquete está disponible sin detener inmediatamente el script se puede utilizar `requireNamespace()`.

```
if (!requireNamespace("ggplot2", quietly = TRUE)) {
  stop("Instala el paquete ggplot2 antes de continuar")
}
```

En las entregas se debe indicar qué paquetes se utilizan y, cuando sea relevante, sus versiones. No es necesario instalar un paquete cada vez que se ejecuta un script.

2.5. Ayuda integrada

La ayuda de R es una herramienta de trabajo, no un recurso secundario. Algunas formas útiles de consultarla son:

```
?sqrt
help(mean)
example(plot)
apropos("linear")
```

También es útil consultar la firma de una función y explorar ejemplos antes de modificar un código encontrado:

```
args(seq)
methods("plot")
example(seq)
```

Cuando aparece un error, conviene leer el mensaje completo, localizar la línea que lo produce y construir un ejemplo mínimo que lo reproduzca. Copiar muchas líneas de código antes de entender el problema suele ocultar la causa.

2.6. Información de la sesión

La versión de R, el sistema operativo y los paquetes cargados pueden afectar a un resultado. Para documentar el entorno se puede guardar la información de la sesión al final de un informe o adjuntarla a una entrega.

```
R.version.string
sessionInfo()
```

2.7. Flujo de trabajo recomendado

Un flujo de trabajo sencillo para cada práctica es:

1. crear o abrir un proyecto para la asignatura;

2. leer el enunciado y escribir una breve descripción del objetivo;
3. guardar el código en un script, ejecutándolo por bloques pequeños;
4. comprobar los resultados con ejemplos sencillos y casos límite;
5. separar datos originales, figuras, resultados y código;
6. redactar la interpretación junto al resultado que la justifica;
7. ejecutar el script desde una sesión limpia antes de entregar.

Entrega de la práctica

La entrega mínima de una práctica debe contener un script ejecutable, los datos necesarios o una referencia precisa a ellos, las figuras generadas y un informe breve que explique el procedimiento, los resultados y las limitaciones. El informe no debe depender de objetos que solo existan en el espacio de trabajo del estudiante.

Observación

Aprender a leer documentación técnica forma parte de la asignatura. En las prácticas se debe consultar la ayuda de R antes de buscar soluciones externas.

Capítulo 3

Objetos, tipos de datos y expresiones

Objetivos de aprendizaje

Al finalizar este capítulo, el estudiante debe ser capaz de:

- identificar el tipo y la estructura básica de un objeto de R;
- utilizar operadores aritméticos, relacionales y lógicos;
- detectar conversiones automáticas y valores especiales;
- inspeccionar objetos antes de operar con ellos y documentar supuestos.

3.1. Objetos y asignación

Un objeto es un nombre asociado a un valor. En R se recomienda usar `<-` para asignar. La asignación crea o actualiza un nombre; la expresión situada a la derecha se evalúa antes de guardar el resultado.

```
radio <- 3
area <- pi * radio^2
area
```

Los nombres deben ser descriptivos. `area_circulo` es preferible a `a` cuando el contexto no es inmediato. En un mismo proyecto conviene elegir una convención y mantenerla: por ejemplo, minúsculas y guiones bajos.

```
radio_m <- 0.25
area_circulo <- pi * radio_m^2

ls()
rm(area_circulo)
```

`ls()` muestra los nombres disponibles y `rm()` elimina objetos. No se recomienda usar `rm(list = ls())` dentro de un script docente: borra el contexto completo y puede ocultar dependencias no declaradas.

3.2. Tipos básicos

```
entero <- 4L
real <- 4.5
texto <- "Laboratorio"
```

```
logico <- TRUE
```

```
class(entero)
class(real)
class(texto)
class(logico)
```

Los tipos atómicos más habituales son `integer`, `double` o numérico, `character`, `logical` y `complex`. La clase que devuelve `class()` es una descripción orientada al uso; `typeof()` informa del tipo interno.

```
typeof(entero)
typeof(real)
typeof(texto)
typeof(logico)

is.numeric(real)
is.character(texto)
is.logical(logico)
```

Para explorar un objeto antes de utilizarlo son especialmente útiles `str()`, `length()` y `names()`.

```
z <- c(2, 4, 6)
str(z)
length(z)
names(z) <- c("a", "b", "c")
names(z)
z
```

3.3. Operadores y precedencia

Las expresiones combinan valores y operadores. Los operadores aritméticos principales son `+`, `-`, `*`, `/`, `^` y `%%`. La división entera se obtiene con `%/%`.

```
17 / 5
17 %/% 5
17 %% 5
2 + 3 * 4
(2 + 3) * 4
```

Las comparaciones producen valores lógicos: `==`, `!=`, `<`, `<=`, `>` y `>=`. Los operadores `&` y `|` combinan condiciones elemento a elemento; `&&` y `||` se reservan para decisiones escalares, especialmente en condicionales.

```
x <- 3
x >= 0 && x <= 10
x == 3 || x == 5
```

Usar paréntesis explícitos hace visible la intención matemática y reduce errores de precedencia.

3.4. Conversión y reciclaje

R puede convertir valores cuando una operación exige un tipo común. En general, los lógicos pueden convertirse en números y los números en texto, pero una conversión implícita puede no coincidir con la intención del programa.

```
as.numeric(TRUE)
as.character(3.5)
as.logical(0)
as.numeric("3.5")
```

La función `as.numeric()` aplicada a texto no numérico produce NA y un aviso. Conviene comprobar siempre el resultado de una conversión.

R también recicla vectores cortos en algunas operaciones. El reciclaje es útil cuando las longitudes son compatibles, pero puede ocultar errores si una longitud no divide a la otra.

```
c(1, 2, 3, 4) + c(10, 100)
c(1, 2, 3) + c(10, 100)
```

En el segundo caso R muestra un aviso. La longitud de los operandos debe formar parte de las comprobaciones de un programa que dependa de posiciones emparejadas.

3.5. Valores especiales y datos ausentes

Los valores NA, NaN, Inf y -Inf no significan lo mismo:

- NA representa un dato ausente o desconocido.
- NaN representa un resultado numéricamente indefinido, como 0/0.
- Inf y -Inf representan infinitos con signo.

```
valores <- c(2, NA, 4, NaN, Inf)
is.na(valores)
is.nan(valores)
is.finite(valores)

mean(valores)
mean(valores, na.rm = TRUE)
```

La opción `na.rm = TRUE` elimina los valores ausentes para esa operación, pero no explica por qué faltan. En un análisis serio hay que distinguir entre eliminar datos, imputarlos o conservar explícitamente la ausencia.

3.6. Errores de redondeo

Los números reales se representan de forma aproximada en el ordenador. Por ello, comparaciones aparentemente obvias pueden fallar.

```
0.1 + 0.2 == 0.3
all.equal(0.1 + 0.2, 0.3)
```

En cálculo computacional conviene distinguir igualdad matemática exacta e igualdad numérica aproximada. Para comparar resultados con tolerancia se puede usar `all.equal()` o una desigualdad explícita.

```
aproximadamente_iguales <- function(x, y, tolerancia = 1e-10) {
  abs(x - y) <= tolerancia
}

aproximadamente_iguales(0.1 + 0.2, 0.3)
```

3.7. Atributos e inspección

Un objeto puede contener atributos adicionales, como nombres, dimensiones o una clase. Estos atributos modifican la forma en que algunas funciones interpretan el valor.

```
temperaturas <- c(18.2, 20.1, 19.7)
names(temperaturas) <- c("lunes", "martes", "miercoles")
attributes(temperaturas)
str(temperaturas)
```

Antes de aplicar una operación a un objeto recibido de otra persona, comprueba al menos su estructura, longitud, nombres y presencia de valores ausentes. Muchas dificultades en R proceden de trabajar con un objeto distinto del que se suponía.

3.8. Lista de comprobación

Para depurar una expresión o preparar una entrega, pregunta:

1. ¿Qué tipo y longitud tiene cada objeto?
2. ¿Las unidades y magnitudes son compatibles?
3. ¿Hay valores ausentes, infinitos o no numéricos?
4. ¿Las comparaciones requieren una tolerancia?
5. ¿La operación está reciclando un vector de forma intencionada?

Capítulo 4

Funciones

4.1. Funciones predefinidas

Una función recibe argumentos y devuelve un resultado. Algunas funciones importantes en las primeras semanas son:

- `sqrt`, `abs`, `round`
- `sin`, `cos`, `tan`
- `exp`, `log`
- `sum`, `prod`
- `mean`, `sd`, `var`
- `length`, `seq`, `rep`

4.2. Funciones propias

```
area_circulo <- function(r) {  
  pi * r^2  
}  
  
area_circulo(1)  
area_circulo(3)
```

Una función debe resolver una tarea concreta. Si una función hace demasiadas cosas, suele ser más difícil de probar y reutilizar.

4.3. Ejemplo: distancia euclídea

```
distancia <- function(x1, y1, x2, y2) {  
  sqrt((x2 - x1)^2 + (y2 - y1)^2)  
}  
  
distancia(0, 0, 3, 4)
```


Capítulo 5

Control de flujo

5.1. Condicionales

Los condicionales permiten que un programa tome decisiones.

```
clasificar_numero <- function(x) {  
  if (x > 0) {  
    "positivo"  
  } else if (x < 0) {  
    "negativo"  
  } else {  
    "cero"  
  }  
}  
  
clasificar_numero(-2)
```

5.2. Bucles

Un bucle repite una operación. En R se usan bucles `for` y `while`; también existen alternativas vectorizadas.

```
suma_cuadrados <- 0  
  
for (k in 1:10) {  
  suma_cuadrados <- suma_cuadrados + k^2  
}  
  
suma_cuadrados
```

5.3. Ejemplo: algoritmo de Euclides

```
mcd <- function(a, b) {  
  a <- abs(a)  
  b <- abs(b)  
  while (b != 0) {  
    resto <- a %% b  
    a <- b  
    b <- resto  
  }  
}
```

```
    a
}
mcd(84, 30)
```

Capítulo 6

Vectores, matrices y estructuras de datos

6.1. Vectores

El vector es una estructura central en R. Muchas operaciones se aplican elemento a elemento.

```
x <- c(1, 2, 3, 4, 5)
x^2
sum(x)
mean(x)
x[x > 3]
```

6.2. Secuencias

```
seq(0, 1, by = 0.1)
seq(0, 2*pi, length.out = 100)
rep(1, times = 5)
```

6.3. Matrices

```
A <- matrix(c(1, 2, 3, 4), nrow = 2, byrow = TRUE)
b <- c(1, 1)

A
t(A)
det(A)
solve(A, b)
```

6.4. Tablas de datos

Un `data.frame` organiza variables en columnas y observaciones en filas.

```
datos <- data.frame(
  estudiante = c("A", "B", "C", "D"),
  practica = c(7.5, 8.0, 6.5, 9.0),
```

```
examen = c(6.0, 7.0, 6.0, 8.5)
)

datos$media <- 0.5 * datos$practica + 0.5 * datos$examen
datos
```

Capítulo 7

Representación gráfica

7.1. Gráficos base

```
x <- seq(-2*pi, 2*pi, length.out = 400)
y <- sin(x)

plot(x, y, type = "l",
     main = "Función seno",
     xlab = "x", ylab = "sin(x)")
abline(h = 0, col = "gray")
```

7.2. ggplot2

El paquete `ggplot2` permite construir gráficos por capas. La idea principal es separar datos, estética y geometría.

```
# install.packages("ggplot2")
library(ggplot2)

datos <- data.frame(
  x = seq(-3, 3, length.out = 200)
)
datos$y <- dnorm(datos$x)

ggplot(datos, aes(x = x, y = y)) +
  geom_line(color = "steelblue", linewidth = 1) +
  labs(title = "Densidad normal estándar",
       x = "x", y = "f(x)") +
  theme_minimal()
```

7.3. Criterios para un buen gráfico

- Debe responder a una pregunta concreta.
- Los ejes deben estar rotulados.
- Las unidades deben indicarse cuando existan.
- El título debe describir el contenido, no decorar.
- El gráfico debe poder interpretarse sin leer el código.

Capítulo 8

Cálculo numérico elemental

8.1. Evaluación de funciones

El ordenador permite evaluar funciones en mallas de puntos y aproximar propiedades globales.

```
f <- function(x) x^3 - x - 1
x <- seq(0, 2, length.out = 200)
y <- f(x)
plot(x, y, type = "l")
abline(h = 0, col = "gray")
```

8.2. Búsqueda de raíces

R incluye `uniroot` para encontrar una raíz en un intervalo donde la función cambia de signo.

```
f <- function(x) x^3 - x - 1
uniroot(f, interval = c(1, 2))
```

8.3. Derivación numérica

Una aproximación simple de la derivada es el cociente incremental centrado:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}.$$

```
derivada_centrada <- function(f, x, h = 1e-5) {
  (f(x + h) - f(x - h)) / (2 * h)
}

derivada_centrada(sin, pi / 4)
cos(pi / 4)
```

8.4. Integración numérica

La regla del trapecio aproxima el área bajo una curva.

```
trapecio <- function(f, a, b, n = 1000) {  
  x <- seq(a, b, length.out = n + 1)  
  y <- f(x)  
  h <- (b - a) / n  
  h * (sum(y) - (y[1] + y[n + 1]) / 2)  
}  
  
trapecio(sin, 0, pi)
```

Capítulo 9

Simulación

9.1. Números pseudoaleatorios

La simulación usa números pseudoaleatorios. Para que un resultado sea reproducible se fija una semilla.

```
set.seed(123)
runif(5)
rnorm(5)
```

9.2. Monte Carlo para estimar π

Una forma clásica de estimar π consiste en lanzar puntos al cuadrado $[-1, 1] \times [-1, 1]$ y contar cuántos caen dentro del círculo unidad.

```
set.seed(1)
n <- 10000
x <- runif(n, -1, 1)
y <- runif(n, -1, 1)
dentro <- x^2 + y^2 <= 1
pi_estimado <- 4 * mean(dentro)
pi_estimado
```

9.3. Error de simulación

La precisión de una estimación de Monte Carlo mejora lentamente. Duplicar la precisión suele requerir multiplicar el número de simulaciones por cuatro.

Capítulo 10

Tratamiento de datos y estadística inicial

10.1. Importación de datos

El trabajo con datos requiere distinguir el archivo original, el código de limpieza y la tabla final. Los datos originales no deben modificarse manualmente.

```
# datos <- read.csv("datos/observaciones.csv")
```

10.2. Resumen descriptivo

```
set.seed(42)
notas <- data.frame(
  grupo = rep(c("A", "B"), each = 20),
  calificacion = c(rnorm(20, 6.5, 1), rnorm(20, 7.2, 1.2))
)

aggregate(calificacion ~ grupo, data = notas, mean)
aggregate(calificacion ~ grupo, data = notas, sd)
```

10.3. Regresión lineal

```
set.seed(10)
x <- 1:30
y <- 2 + 0.7 * x + rnorm(30, sd = 2)

modelo <- lm(y ~ x)
summary(modelo)
plot(x, y)
abline(modelo, col = "red")
```

La salida de un modelo no debe copiarse sin interpretación. En un informe conviene explicar qué representa cada coeficiente, qué variabilidad queda sin explicar y qué limitaciones tiene el modelo.

Capítulo 11

Edición científica con LaTeX

11.1. Estructura mínima

```
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage{amsmath,amssymb}

\title{Mi primer documento}
\author{Nombre del estudiante}
\date{\today}

\begin{document}
\maketitle

\section{Introducción}
Este documento contiene texto y fórmulas.

\[
\int_0^1 x^2 dx = \frac{1}{3}.
\]

\end{document}
```

11.2. Fórmulas

LaTeX separa las fórmulas en línea, como $a^2 + b^2 = c^2$, de las fórmulas destacadas:

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}.$$

11.3. Tablas

```
\begin{tabular}{lrr}
\toprule
Método & Error & Tiempo \\
\midrule
A & 0.013 & 0.42 \\
B & 0.008 & 0.71 \\
\bottomrule
\end{tabular}
```

```
\end{tabular}
```

11.4. Buenas prácticas

- Separar contenido, estructura y formato.
- Usar etiquetas y referencias cruzadas para ecuaciones, figuras y tablas.
- Evitar capturas de pantalla de código o resultados cuando pueden incluirse como texto.
- Compilar con frecuencia para detectar errores pronto.

Capítulo 12

Markdown, R Markdown y Quarto

12.1. Markdown

Markdown permite escribir texto estructurado con una sintaxis ligera.

```
# Título

Un párrafo con negrita y una fórmula en línea:  $x^2$ .

- Primer elemento
- Segundo elemento
```

12.2. Cuadernos reproducibles

Un cuaderno reproducible combina texto, código y resultados. La idea central es que el informe se genere desde el código, no que el estudiante copie manualmente salidas de consola.

```
---
title: "Informe de práctica"
format: pdf
---

## Cálculo

```{r}
x <- seq(0, 1, length.out = 100)
mean(x^2)
```
```

12.3. Estructura recomendada de un informe

1. Pregunta o problema.
2. Método computacional.
3. Código esencial.
4. Resultados.
5. Interpretación.
6. Limitaciones o comprobaciones.

Parte I

Prácticas de laboratorio

Capítulo 13

Guía general de prácticas

13.1. Formato común de entrega

Cada práctica debe entregarse como carpeta comprimida o repositorio con:

- un script principal `practicaXX.R`;
- los datos usados, si procede;
- un informe breve en PDF, HTML o Quarto;
- las figuras generadas por el código, si no están incrustadas en el informe.

13.2. Rúbrica común

| Criterio | Peso | Evidencia esperada |
|-----------------------|------|--|
| Corrección matemática | 30 % | Resultados coherentes, comprobaciones y uso adecuado de conceptos. |
| Código | 30 % | Claridad, nombres descriptivos, funciones cuando proceda, ausencia de duplicación innecesaria. |
| Reproducibilidad | 20 % | Proyecto organizado, rutas relativas, semilla aleatoria cuando proceda, informe generado desde código. |
| Comunicación | 20 % | Explicación breve, gráficos legibles, interpretación de resultados. |

Capítulo 14

Práctica 1. Entorno de trabajo y primer proyecto

Objetivos de aprendizaje

- Crear un proyecto de RStudio/Posit.
- Organizar carpetas de trabajo.
- Ejecutar comandos básicos en consola y script.
- Guardar un primer informe de resultados.

14.1. Actividad

1. Crear una carpeta de proyecto con subcarpetas `datos`, `scripts`, `figuras` e `informes`.
2. Escribir un script que calcule áreas y volúmenes de figuras elementales.
3. Incluir al menos una función propia.
4. Guardar los resultados principales en un archivo de texto o informe breve.

```
area_rectangulo <- function(base, altura) {  
  base * altura  
}  
  
volumen_esfera <- function(radio) {  
  4 / 3 * pi * radio^3  
}  
  
area_rectangulo(3, 5)  
volumen_esfera(2)
```

Entrega de la práctica

Entregar el proyecto organizado y un informe de una página con los comandos usados, resultados y una reflexión sobre la estructura de carpetas.

Capítulo 15

Práctica 2. Cálculo elemental y precisión numérica

Objetivos de aprendizaje

- Trabajar con expresiones numéricas.
- Reconocer errores de redondeo.
- Comparar resultados exactos y aproximados.

15.1. Actividad

1. Evaluar expresiones con potencias, raíces, logaritmos y trigonometría.
2. Comprobar identidades matemáticas con valores numéricos.
3. Investigar casos donde la comparación exacta falla por precisión finita.
4. Escribir una función `casi_igual(x, y, tol)`.

```
casi_igual <- function(x, y, tol = 1e-8) {  
  abs(x - y) < tol  
}
```

```
casi_igual(0.1 + 0.2, 0.3)
```

Entrega de la práctica

Entregar un script y un informe con tres ejemplos de identidades verificadas numéricamente y una explicación de las discrepancias observadas.

Capítulo 16

Práctica 3. Funciones propias

Objetivos de aprendizaje

- Diseñar funciones con argumentos claros.
- Probar funciones con casos simples.
- Documentar el comportamiento esperado.

16.1. Actividad

Programar funciones para:

1. convertir grados a radianes y radianes a grados;
2. calcular la distancia entre dos puntos del plano;
3. calcular el valor de un polinomio en un punto;
4. normalizar un vector dividiendo por su norma euclídea.

```
evaluar_polinomio <- function(coeficientes, x) {  
  potencias <- 0:(length(coeficientes) - 1)  
  sum(coeficientes * x^potencias)  
}
```

```
evaluar_polinomio(c(1, 0, 3), 2)
```

Entrega de la práctica

Entregar código con pruebas de cada función y un breve comentario sobre posibles entradas no válidas.

Capítulo 17

Práctica 4. Condicionales, bucles y algoritmos elementales

Objetivos de aprendizaje

- Implementar algoritmos con control de flujo.
- Distinguir bucles `for` y `while`.
- Incorporar comprobaciones de entrada.

17.1. Actividad

1. Implementar el algoritmo de Euclides para el máximo común divisor.
2. Programar una función que determine si un entero es primo.
3. Calcular los primeros n términos de la sucesión de Fibonacci.
4. Comparar el tiempo de ejecución para distintos tamaños de entrada.

```
es_primo <- function(n) {  
  if (n < 2) return(FALSE)  
  if (n == 2) return(TRUE)  
  for (k in 2:floor(sqrt(n))) {  
    if (n %% k == 0) return(FALSE)  
  }  
  TRUE  
}
```

Entrega de la práctica

Entregar funciones, pruebas y una tabla con resultados para varios valores de n .

Capítulo 18

Práctica 5. Vectores y matrices

Objetivos de aprendizaje

- Usar operaciones vectorizadas.
- Resolver sistemas lineales pequeños.
- Interpretar resultados matriciales.

18.1. Actividad

1. Crear vectores mediante `c`, `seq` y `rep`.
2. Calcular normas, productos escalares y distancias.
3. Construir matrices y operar con transpuestas, determinantes e inversas.
4. Resolver sistemas $Ax = b$ con `solve`.

```
A <- matrix(c(2, 1, 1, 3), nrow = 2)
b <- c(1, 4)
x <- solve(A, b)
A %*% x
```

Entrega de la práctica

Entregar un informe con al menos tres sistemas lineales y una verificación $Ax = b$ para cada uno.

Capítulo 19

Práctica 6. Datos en tablas

Objetivos de aprendizaje

- Crear y manipular `data.frame`.
- Calcular nuevas columnas.
- Filtrar observaciones.

19.1. Actividad

1. Crear una tabla con datos simulados de estudiantes y calificaciones.
2. Calcular nota final ponderada.
3. Clasificar a los estudiantes como apto/no apto.
4. Obtener medias por grupo.

```
set.seed(5)
datos <- data.frame(
  grupo = rep(c("A", "B", "C"), each = 10),
  practica = runif(30, 4, 10),
  examen = runif(30, 3, 10)
)

datos$final <- 0.5 * datos$practica + 0.5 * datos$examen
datos$apto <- datos$final >= 5
aggregate(final ~ grupo, data = datos, mean)
```

Entrega de la práctica

Entregar tabla final, código y comentario sobre la distribución de calificaciones por grupo.

Capítulo 20

Práctica 7. Visualización de funciones y datos

Objetivos de aprendizaje

- Representar funciones matemáticas.
- Construir gráficos de dispersión e histogramas.
- Mejorar títulos, ejes y leyendas.

20.1. Actividad

1. Representar $f(x) = \sin(x)$, $g(x) = \cos(x)$ y $h(x) = e^{-x^2}$.
2. Crear un histograma de una muestra normal.
3. Crear un diagrama de dispersión con recta de ajuste.
4. Exportar al menos una figura a la carpeta **figuras**.

```
png("figuras/seno_coseno.png", width = 900, height = 600)
x <- seq(-2*pi, 2*pi, length.out = 400)
plot(x, sin(x), type = "l", col = "blue", ylab = "valor")
lines(x, cos(x), col = "red")
legend("topright", legend = c("sen(x)", "cos(x)"),
      col = c("blue", "red"), lty = 1)
dev.off()
```

Entrega de la práctica

Entregar script, figuras generadas y comentario sobre qué gráfico comunica mejor cada tipo de información.

Capítulo 21

Práctica 8. Raíces y aproximación numérica

Objetivos de aprendizaje

- Resolver ecuaciones no lineales sencillas.
- Usar `uniroot`.
- Comparar aproximaciones con soluciones conocidas.

21.1. Actividad

1. Representar $f(x) = x^3 - x - 1$ en varios intervalos.
2. Localizar un intervalo con cambio de signo.
3. Calcular una raíz con `uniroot`.
4. Implementar el método de bisección como función propia.

```
biseccion <- function(f, a, b, tol = 1e-8, max_iter = 100) {  
  if (f(a) * f(b) > 0) stop("No hay cambio de signo")  
  for (i in 1:max_iter) {  
    c <- (a + b) / 2  
    if (abs(f(c)) < tol || (b - a) / 2 < tol) return(c)  
    if (f(a) * f(c) < 0) {  
      b <- c  
    } else {  
      a <- c  
    }  
  }  
  (a + b) / 2  
}
```

Entrega de la práctica

Comparar `biseccion` y `uniroot` en tres funciones distintas.

Capítulo 22

Práctica 9. Derivación e integración numérica

Objetivos de aprendizaje

- Aproximar derivadas mediante diferencias finitas.
- Aproximar integrales mediante reglas de cuadratura.
- Analizar el efecto del tamaño de paso.

22.1. Actividad

1. Implementar diferencias hacia delante y centradas.
2. Aproximar derivadas de $\sin(x)$, e^x y x^3 .
3. Implementar la regla del trapecio.
4. Estudiar cómo cambia el error al variar h o n .

Entrega de la práctica

Entregar una tabla de errores y un gráfico que muestre la relación entre precisión y tamaño de paso.

Capítulo 23

Práctica 10. Simulación y Monte Carlo

Objetivos de aprendizaje

- Fijar semillas aleatorias.
- Simular experimentos.
- Estimar probabilidades y magnitudes geométricas.

23.1. Actividad

1. Simular lanzamientos de una moneda equilibrada.
2. Simular tiradas de dos dados y estimar la probabilidad de suma 7.
3. Estimar π mediante Monte Carlo.
4. Repetir el experimento con distintos tamaños muestrales.

Entrega de la práctica

Entregar resultados para al menos cuatro tamaños muestrales y explicar la variabilidad observada.

Capítulo 24

Práctica 11. Estadística descriptiva

Objetivos de aprendizaje

- Calcular medidas de posición y dispersión.
- Visualizar distribuciones.
- Comparar grupos.

24.1. Actividad

1. Generar o importar una tabla de datos con una variable cuantitativa y una variable de grupo.
2. Calcular media, mediana, desviación típica y rango intercuartílico.
3. Construir histogramas y diagramas de caja.
4. Redactar una comparación de grupos.

Entrega de la práctica

Entregar un informe con tablas, gráficos y una interpretación de máximo dos páginas.

Capítulo 25

Práctica 12. Regresión lineal

Objetivos de aprendizaje

- Ajustar un modelo lineal simple.
- Interpretar pendiente e intercepto.
- Diagnosticar visualmente el ajuste.

25.1. Actividad

1. Simular datos con relación lineal y ruido.
2. Ajustar $\text{lm}(y \sim x)$.
3. Dibujar la nube de puntos y la recta ajustada.
4. Analizar residuos mediante gráficos.

```
set.seed(20)
x <- runif(50, 0, 10)
y <- 1.5 + 2.2 * x + rnorm(50, sd = 2)
modelo <- lm(y ~ x)
summary(modelo)
plot(modelo)
```

Entrega de la práctica

Entregar informe reproducible con interpretación de los coeficientes y comentario sobre la calidad del ajuste.

Capítulo 26

Práctica 13. Documento científico en LaTeX

Objetivos de aprendizaje

- Crear un documento LaTeX desde cero.
- Escribir fórmulas, tablas y referencias cruzadas.
- Compilar un PDF correctamente.

26.1. Actividad

1. Crear un documento con título, autor, resumen y dos secciones.
2. Incluir al menos tres fórmulas matemáticas.
3. Incluir una tabla con resultados numéricos.
4. Usar etiquetas y referencias cruzadas.
5. Añadir una bibliografía mínima.

Entrega de la práctica

Entregar el archivo `.tex`, el PDF compilado y cualquier archivo auxiliar necesario para la bibliografía.

Capítulo 27

Práctica 14. Informe reproducible con R Markdown o Quarto

Objetivos de aprendizaje

- Combinar texto, código y resultados.
- Generar un informe reproducible.
- Evitar copiar manualmente resultados.

27.1. Actividad

1. Crear un archivo `.qmd` o `.Rmd`.
2. Incluir una introducción al problema.
3. Ejecutar código R dentro del documento.
4. Mostrar al menos un gráfico generado desde el propio informe.
5. Renderizar a HTML o PDF.

Entrega de la práctica

Entregar el archivo fuente y el informe renderizado. El informe debe poder regenerarse sin cambios manuales.

Capítulo 28

Práctica 15. Proyecto integrador

Objetivos de aprendizaje

- Integrar programación, datos, gráficos, cálculo y comunicación científica.
- Trabajar con una estructura de proyecto completa.
- Presentar resultados de forma defendible.

28.1. Propuesta

El estudiante elegirá uno de los siguientes proyectos:

1. Estudio numérico de una familia de funciones y sus raíces.
2. Simulación de un experimento aleatorio y estimación de probabilidades.
3. Análisis descriptivo y gráfico de un conjunto de datos.
4. Ajuste e interpretación de un modelo lineal simple.

28.2. Requisitos mínimos

- Código R organizado en al menos un script principal.
- Al menos una función propia.
- Al menos dos gráficos interpretados.
- Informe final en LaTeX, R Markdown o Quarto.
- Conclusiones que conecten los resultados con el problema planteado.

Entrega de la práctica

Entregar proyecto completo y realizar una defensa breve de 5 minutos. La defensa debe explicar el problema, el método, los resultados y una limitación del trabajo.

Apéndice A

Referencia rápida de R

A.1. Objetos y operaciones

```
x <- 3
y <- c(1, 2, 3)
z <- seq(0, 1, length.out = 5)
sum(y)
mean(y)
length(y)
```

A.2. Funciones

```
nombre_funcion <- function(argumento1, argumento2) {
  resultado <- argumento1 + argumento2
  resultado
}
```

A.3. Control de flujo

```
if (condicion) {
  # instrucciones
} else {
  # instrucciones alternativas
}

for (k in 1:10) {
  print(k)
}
```

Apéndice B

Plantilla de informe de práctica

Título de la práctica

Nombre:

Fecha:

1. Problema

Descripción breve del problema que se quiere resolver.

2. Método

Explicación de las funciones, algoritmos o datos usados.

3. Código esencial

Fragmentos principales o referencia al script.

4. Resultados

Tablas, valores numéricos y gráficos.

5. Interpretación

Qué significan los resultados y qué limitaciones tienen.

Apéndice C

Bibliografía inicial

- Matloff, N. *The Art of R Programming*. No Starch Press.
- Wickham, H. *Advanced R*. Chapman and Hall/CRC.
- Wickham, H. y Golemund, G. *R for Data Science*. O'Reilly Media.
- Grätzer, G. *More Math Into LaTeX*. Springer.
- Xie, Y., Dervieux, C. y Riederer, E. *R Markdown Cookbook*. Chapman and Hall/CRC.
- Venables, W. N., Smith, D. M. y R Core Team. *An Introduction to R*.
- Lamport, L. *LaTeX: A Document Preparation System*. Addison-Wesley.
- Xie, Y., Allaire, J. J. y Golemund, G. *R Markdown: The Definitive Guide*. Chapman and Hall/CRC.

Notas para revisión docente

Este borrador se ha preparado a partir de la guía docente disponible. Antes de usarlo como manual definitivo conviene revisar:

- calendario académico real y festivos;
- criterios exactos de evaluación aprobados por la coordinación;
- software disponible en aulas;
- conjuntos de datos oficiales de la asignatura;
- formato institucional exigido para guiones de prácticas;
- nivel inicial real del grupo.